HYUNDAI AI Insight Campus · 온디바이스 AI

Day 1 라즈베리파이 설정과 I/O 지도

Embedded I/O 제어 1/5 · 10/13(화) · Raspberry Pi 4 + Raspberry Pi OS Lite 64-bit (Trixie)

강의 자료 · **애니메이션** · **워크시트**

Raspberry Pi 4 — 리눅스가 도는 작은 컴퓨터

4 코어

Cortex-A72 1.5GHz
(BCM2711)

4GB

LPDDR4 메모리

40 핀

GPIO 헤더
(코드는 BCM 번호)

3.3V

GPIO 전압
5V 내성 없음

MCU 와 다른 점

ADC 없음 → Day 4 에 PCF8591
하드웨어 PWM 2 채널 (GPIO18 · 19)
핀 접근은 커널 드라이버 → 장치 파일

이 과목에서 쓰는 것

Raspberry Pi OS Lite 64-bit (Trixie)
USB 첫 접속 → Wi-Fi · C · libgpiod 2.x
KS3014 키트 + GPIO Extension Board

5 일 동안 만드는 것

Day 1

라즈베리파이 설정

`check_env.sh`

Day 2

GPIO 출력 · 입력 · 이벤트

`react.c`

Day 3

PWM · 모터 · 타이밍

`alarm.c`

Day 4

I2C · SPI · UART

`gateway.c`

Day 5

커널 드라이버 · 종합

`envmon` 서비스

각 Day 의 미니 프로젝트 코드는 Day 5 스마트 환경 모니터의 모듈로 다시 쓰인다 .

오늘의 8 교시

1
09:00
과정 소개 · Pi 4 vs MCU

2
10:00
하드웨어 준비 · 전기적 주의

3
11:00
OS 설치 · USB 첫 부팅

4
13:00
USB 접속 · Wi-Fi 설정

5
14:00
인터페이스 · Device Tree

6
15:00
개발 환경 · 권한

7
16:00
장치 파일로 보는 I/O 지도

8
17:00
버튼 입력 · check_env.sh

점심 12:00–13:00 · 교시당 50 분

ESP32 와 Raspberry Pi 4

1 교시

항목	ESP32 (MCU 과목)	Raspberry Pi 4 (이 과목)
운영체제	없음 (FreeRTOS 정도)	Linux
프로그램 올리기	PC 에서 빌드 → USB 업로드	보드 안에서 gcc 로 빌드
전원 → 내 코드	1 초 미만	약 20~40 초 (부팅)
핀 접근	레지스터 · Arduino API 직접	커널 드라이버 → 장치 파일
아날로그 입력	ADC 내장 (12-bit)	없음 → PCF8591 (Day 4)
하드웨어 PWM	LEDC 16 채널	2 채널 (GPIO18 · 19)
실시간성	µs 예측 가능	스케줄러 지연 존재

전원을 넣고 내 프로그램이 핀을 만지기까지

1교시



ESP32 : 전원 → ROM 부트로더 → setup() / loop() (레지스터 직접)

config.txt 는 한 번만 읽힌다

부팅 초기에 GPU 펌웨어가 읽어 Device Tree 를 만든다 .
고치면 반드시 재부팅 .

장치 파일이 생겨야 시작

드라이버가 /dev/gpiochip0 을 만든 뒤에야 사용자 프로그램이 핀에 닿는다 .

애니메이션 D1-1 • ESP32 vs Pi 부팅 ▶

40 핀 헤더 — 물리 번호와 BCM 번호

2 교시



위 줄 = 짝수 핀 2 · 4 · 6 … 아래 줄 = 홀수 핀 1 · 3 · 5 … 왼쪽 아래가 1 번 (네모 패드)

물리	이름	오늘 쓰는 곳
11	GPIO17	LED (7 교시)
29	GPIO5	버튼 (8 교시)
9	GND	브레드보드 GND 레일

코드는 BCM 번호

gpioset -c gpiochip0 17=1
→ 물리 11 번 핀

대체 기능 (PWM · I2C · SPI · UART) 은 Device Tree 와 드라이버가 정한다

애니메이션 D1-2 • 40핀 헤더 탐색기 ▶

전기적 주의 — MCU 과목보다 엄격하다

2 교시

3.3V 전용

GPIO 에 5V 를 넣으면 SoC 가 망가진다. 보드 교체 외에 방법이 없다.

핀당 16mA

LED 에는 반드시 220Ω 직렬. 모터는 드라이버 칩 · 릴레이를 거친다.

5V 단락 금지

물리 2 · 4 번 (5V) 은 3V3 · GPIO 바로 옆. 배선 변경은 poweroff 후.

5V 출력은 분압

HC-SR04 Echo 처럼 5V 를 내는 센서는 저항 분압 후 연결 (Day 3).

GPIO Extension Board 연결

2 교시

1. **빨간 줄을 1 번 핀 (3V3) 쪽에**
   40-Pin 케이블 방향이 틀리면 이름표가 모두 어긋난다
2. **케이블 반대쪽을 Extension Board 에**
   홈 방향이 맞아야 들어간다
3. **브레드보드 가운데 홈에 걸쳐 꽂기**
   한쪽에 꽂으면 양쪽 핀이 같은 열 → 단락
4. **GND 를 파란 레일 (–) 에**
   오늘 모든 부품의 GND 가 모이는 곳
5. **짝과 서로 확인 후 전원**
   거꾸로면 3V3 자리에 5V 가 나온다

오늘의 배선

GPIO17 (물리 11)
→ 220Ω → LED(+)
LED(–) → GND 레일

GPIO5 (물리 29)
→ 버튼 → GND 레일
(내부 pull-up, 저항 없음)

모니터 없이 — USB 로 먼저 , Wi-Fi 는 나중에

3 교시

1 단계 · 3~4 교시

PC ═ USB-C 데이터 케이블 ═ Pi

전원 + 네트워크를 한 줄로
PC 와 Pi 둘만의 망 (Pi = 10.12.194.1)

2 단계 · 4 교시 끝 ~

PC — 강의장 Wi-Fi — Pi

전원은 5V 3A 어댑터
40 대가 한 망 → 이름은 좌석 번호

Imager 설정	입력값	빠뜨리면
OS	Lite (64-bit) · Trixie · 2025-10-20 이후	gadget 패키지 없음 · libgpiod 1.6
호스트 이름	rpi-01 ~ rpi-40 (좌석 번호)	Wi-Fi 에서 남의 보드에 접속
사용자	강사 안내값	로그인할 계정 없음
무선 LAN	비워 둠 (4 교시 nmcli)	—
SSH	켜기 (비밀번호 인증)	Connection refused

애니메이션 D1-3 • SD 굽기 → USB 첫 접속 ▶

첫 부팅 전 user-data, 그리고 PC USB 로 첫 부팅

3 교시

```
# bootfs/user-data 맨 끝
rpi:
  enable_usb_gadget: true
```

확인할 것	이유
스페이스 2 칸 (탭 금지)	YAML 오류면 user-data 전체 무시
rpi: 가 이미 있으면 한 번만	키 중복 금지
Pi 에 꽂기 전에	첫 부팅에만 적용 (cloud-init)

첫 부팅	확인
Pi USB-C ↔ PC 본체 USB (데이터 케이블)	어댑터는 쓰지 않음 — 케이블이 전원
PWR 깜빡임 · 재부팅 반복	PC 포트 전력 부족 → 뒷면 포트로
불은 켜지는데 USB 장치 없음	충전 전용 케이블 의심
2~3 분 , 자동 재부팅 1 회	정상

Windows 드라이버 : github.com/raspberrypi/rpi-usb-gadget/releases → rpi-usb-gadget-driver-setup.exe (PC 당 한 번)

USB 접속 — PuTTY 로 첫 로그인

4 교시



화면에 보이는 증상	멈춘 곳	해결
ncpa.cpl 에 새 어댑터가 없음	USB-C 케이블	데이터 케이블 · user-data 확인 · 3 분 대기
장치 관리자 " 알 수 없는 장치 (RNDIS)"	usb0	rpi-usb-gadget-driver-setup.exe 설치
rpi-07.local 을 찾을 수 없음	주소	Host Name 에 10.12.194.1
Connection refused	sshd	Imager 에서 SSH 켜고 다시 굽기
접속 중 끊기고 재부팅	전원	PC 본체 뒷면 포트에 직접

애니메이션 D1-4 • USB • Wi-Fi 접속 진단 ▶

40 명이 같은 이름 · 같은 IP 를 써도 되나

4 교시

USB 망 — 충돌 없음

PC 1 대 ↔ Pi 1 대 , 케이블 안에서만 .
40 대 모두 10.12.194.1 이어도 서로 보이지 않는다 .

Wi-Fi 망 — 충돌

40 대가 한 망 . 먼저 켜진 보드가 rpi-07.local 을 갖고 ,
나중 보드는 rpi-07-2.local 로 밀린다 → 남의 보드에 접속 .

```
hostname              #  좌석 번호인지 확인
sudo raspi-config nonint \
     do_hostname rpi-07
sudo reboot
```

이름은 처음부터 좌석 번호로 . 틀렸다면 Wi-Fi 에 붙기 전 , USB 로 접속한 지금 고친다 . 보드 케이스에 같은 번호 스티커 .

Wi-Fi 설정 — PuTTY 안에서 nmcli

4 교시

```
# ① 국가 코드부터 (없으면 rfkill 차단)
sudo raspi-config nonint do_wifi_country KR

# ② 검색
sudo nmcli dev wifi rescan
nmcli dev wifi list

# ③ 접속 (저장 → 재부팅 후 자동 연결)
sudo nmcli dev wifi connect "SSID" password "PW"

# ④ 확인
nmcli dev status
ip -4 addr show wlan0
```

AP 목록이 비어 있음

국가 코드 설정 후 sudo reboot

Secrets were required

Wi-Fi 비밀번호 오류 — 대소문자 확인

붙은 뒤 바로

sudo apt update && sudo apt full-upgrade -y

Wi-Fi 접속으로 갈아타기

4 교시

1. **USB 꽂은 채 sudo reboot**
   wlan0 이 자동으로 다시 붙는지 확인

2. **PC 를 같은 Wi-Fi 에**
   —

3. **PuTTY 새 세션 = wlan0 IP**
   USB 가 꽂힌 동안 .local 은 USB 쪽으로 풀릴 수 있다

4. **sudo rpi-usb-gadget off → poweroff**
   Wi-Fi 접속이 확인된 뒤에만 ! 먼저 끄면 들어갈 길이 없다

5. **5V 3A 어댑터로 부팅 → rpi-07.local**
   vcgencmd get_throttled = 0x0

왜 어댑터로 바꾸나

PC USB 전원은 빠듯하다 — throttled 저전압 기록 .

Day 3 의 모터 · 스테퍼는 PC 포트로 감당할 수 없다 .

5 교시부터는 Wi-Fi + 어댑터 전원으로 진행 .

인터페이스 켜기 — raspi-config 와 config.txt

5 교시

```
# 0 = 켜기, 1 = 끄기
sudo raspi-config nonint do_i2c 0
sudo raspi-config nonint do_spi 0
sudo raspi-config nonint do_serial_hw 0
sudo raspi-config nonint do_serial_cons 1
```

```
# /boot/firmware/config.txt [all] 아래
dtoverlay=pwm-2chan
dtoverlay=disable-bt

sudo systemctl disable hciuart
sudo reboot
```

줄 끝 주석 금지 dtoverlay=pwm-2chan # PWM → 주석까지 overlay 이름으로 읽혀 적용되지 않는다. 주석은 별도 줄에.

재부팅 후 확인	기대 결과
ls /dev/i2c-1 /dev/spidev0.0	둘 다 있음
ls /sys/class/pwm	pwmchip0
ls -l /dev/serial0	→ ttyAMA0

애니메이션 D1-5 • config.txt → /dev ▶

Device Tree 와 overlay — 하드웨어 설명서

5 교시

bcm2711-rpi-4-b.dtb

보드 기본 설명서
(I2C · SPI 는 꺼져 있음)

+ dtparam

기존 장치의 설정을 바꿈
i2c_arm=on · spi=on

+ dtoverlay

조각을 덧붙임 (.dtbo)
pwm-2chan · disable-bt

최종 Device Tree

→ 커널 → 드라이버
→ /dev · /sys

x86 PC 는 BIOS/ACPI 가 하드웨어를 알려 주지만 , ARM 보드는 Device Tree 를 커널에 넘긴다 . 설명서에 없는 장치에는 드라이버가 붙지 않는다 .

Day 5 에서 overlay 를 직접 작성해 LED · 버튼을 커널 장치로 만든다 . 모든 overlay 설명 : /boot/firmware/overlays/README

왜 disable-bt 인가 — PL011 과 mini UART

5 교시

UART	클럭	기본 연결	특징
PL011 (ttyAMA0)	독립 클럭	Bluetooth	보율이 정확
mini UART (ttyS0)	CPU 코어 클럭	GPIO14 · 15	클럭이 바뀌면 보율이 흔들림

disable-bt

블루투스를 끄고 PL011 을 GPIO14 · 15 로 돌려받는다 . Day 4 ESP32 통신의 전제 .

/dev/serial0

" 지금 GPIO 에 연결된 UART" 를 가리키는 별명 . ttyAMA0 이면 OK, ttyS0 이면 config.txt 재확인 .

개발 환경 — libgpiod 는 반드시 2.x

6 교시

```
sudo apt install -y build-essential \
     gpiod libgpiod-dev i2c-tools

pkg-config --modversion libgpiod   # 2.2.1
```

패키지	쓰는 곳
build-essential	gcc · make
gpiod	gpioset · gpioinfo · gpiomon
libgpiod-dev	gpiod.h (Day 2~5)
i2c-tools	i2cdetect (Day 4)

v1 과 v2 는 C API 가 완전히 다르다
인터넷 예제 대부분은 v1 (gpiod_line_request_output ...)
v2 예제에는 gpiod_request_config 같은 이름이 있다

```
# 공통 Makefile 핵심
LDLIBS = $(shell pkg-config \
  --libs libgpiod) -lpthread
%: %.c
[ 탭 ]$(CC) $(CFLAGS) -o $@ $< $(LDLIBS)
```

장치 파일 권한 — 열쇠는 그룹

6 교시

```
crw-rw----  1  root  gpio  254, 0  Oct 13 14:41  /dev/gpiochip0
```

c

문자 장치
바이트 단위로 읽고 씀

rw-rw----

소유자 · 그룹만
읽기 · 쓰기

gpio

이 그룹이면
sudo 없이 사용

254, 0

주 번호 (드라이버)
부 번호 (몇 번째)

Imager 로 만든 사용자는 gpio · i2c · spi · dialout 그룹에 이미 들어 있다 . 하드웨어 프로그램은 sudo 로 실행하지 않는다 . 그룹을 추가했다면 다시 로그인 .

애니메이션 D1-6 • 장치 파일 권한과 그룹 ▶

Linux 에서 하드웨어는 파일이다

7 교시

하드웨어	장치 파일	드라이버	Day
GPIO	/dev/gpiochip0	pinctrl-bcm2711	2 · 3 · 5
하드웨어 PWM	/sys/class/pwm/pwmchip0	pwm-bcm2835	3
I2C	/dev/i2c-1	i2c-bcm2835	4
SPI	/dev/spidev0.0	spi-bcm2835 · spidev	4
UART	/dev/ttyAMA0 (serial0)	amba-pl011	4
DHT11	/sys/bus/iio/devices/...	dht11	5

드라이버만 레지스터를 만지고 , 내 프로그램은 open · read · write · ioctl 로 부탁한다 . sysfs GPIO(/sys/class/gpio) 는 폐기 — 번호가 바뀌고 (17 → 529) 해제가 안 남는다 .

먼저 요청한 프로세스가 라인을 가진다

7 교시

```
# 창 1
gpioset -c gpiochip0 17=1
(라인을 쥐고 기다림)
^C   ← 커널이 라인 해제
```

```
# 창 2
gpioinfo -c gpiochip0 17
  … consumer="gpioset"
gpioset -c gpiochip0 17=0
  → Device or resource busy
```

소유권

같은 라인을 두 번 요청하면 EBUSY. MCU 에서는 아무도 막지 않던 충돌.

자동 해제

Ctrl+C · kill -9 로 프로세스가 끝나면 fd 가 닫히고 커널이 정리한다.

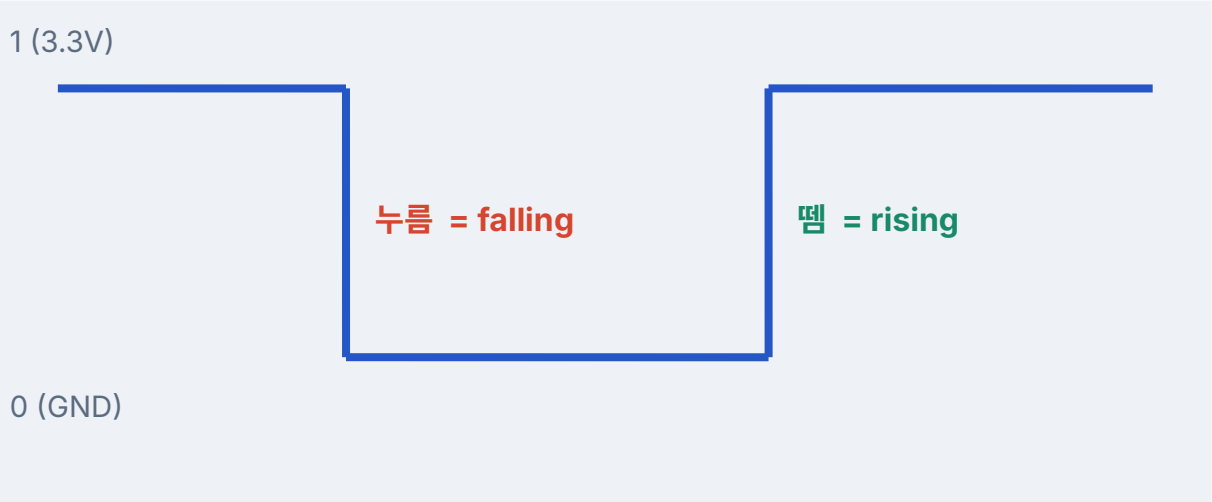
드라이버도 소유자

GPIO8 은 spi0 CS0 이 쥐고 있어 busy. 버스 핀은 일반 GPIO 로 쓰지 않는다.

애니메이션 D1-7 • 라인 요청과 busy ▶

버튼은 누르면 0 — pull-up 과 엣지

8 교시




```
gpioget -c gpiochip0 -b pull-up 5
  "5"=active  ( 안 누름 )

gpiomon -c gpiochip0 -b pull-up \
        -e falling -p 10ms 5
```

bias 는 필수

연결 없는 입력은 floating — 값이 제멋대로 . GND 쪽 버튼에는 pull-up.

바운싱

접점이 몇 ms 떨려 엣지가 여러 번 . -p 10ms 디바운스로 해결 .

커널 타임스탬프

gpiomon 맨 앞 숫자 . Day 2 반응속도 게임의 ms 측정 근거 .

애니메이션 D1-8 • pull-up 버튼과 엣지 ▶

미니 프로젝트 — check_env.sh

8 교시

점검 항목	[FAIL] 이면	고친 교시
보드 · OS · 커널	Trixie · aarch64 아님	3 교시
전원 · 온도	(WARN) throttled ≠ 0x0	4 교시
이름 · Wi-Fi · USB	wlan0 미연결 (이름 · gadget 은 WARN)	4 교시
장치 파일 · serial0 · 콘솔	/dev 노드 없음 · ttyS0	5 교시
그룹	gpio · i2c · spi · dialout 누락	6 교시
도구 · libgpiod	1.x 이거나 없음	6 교시

```
cd ~/eio/day1/src
bash check_env.sh
…
결과 : OK 25 개 ,
       FAIL 0 개
Day 2 준비 완료
```

모두 [OK] → 결과 화면 캡처 제출 . 도전 : FAIL 이면 종료 코드 1 · 통과하면 LED 2 초 깜빡임 · 버튼 점검 항목 추가

오늘 정리

config.txt 는 부팅 때 한 번 · 하드웨어는 장치 파일 · 그룹이 열쇠 · 라인은 요청한 프로세스가 소유 · 입력은 pull-up 과 falling

다음 시간 (Day 2, 10/14) libgpiod v2 C API · /dev/gpiomem 레지스터 비교 · poll() · epoll · 반응속도 게임 . 오늘 배선한 GPIO17 LED 와 GPIO5 버튼은 그대로 둔다 .

워크시트 풀기 ▶